

H2 Database: Sistema de Gestión de Bases de Datos en Java

Una guía completa sobre H2 Database, un sistema de gestión de bases de datos relacionales de código abierto escrito en Java, ideal para desarrollo y pruebas.



H2

Agenda

¿Qué es H2 Database?

Introducción y características principales

Modos de Operación

En memoria, basado en archivos, embebido y cliente-servidor

Instalación

Cómo descargar H2 sin gestores de dependencias

Configuración con JPA

Integración con Jakarta Persistence API

¿Qué es H2 Database?

H2 es un sistema de gestión de bases de datos relacionales (RDBMS) de código abierto, completamente escrito en Java. Se distingue por su ligereza, velocidad y facilidad de uso, lo que la convierte en una opción muy popular para entornos de desarrollo, pruebas unitarias e integración de aplicaciones Java.



Características Clave de H2



100% Java

Al estar escrita en Java, H2 es altamente portable y puede ejecutarse en cualquier plataforma con una JVM compatible.



Consola Web Integrada

Interfaz de usuario basada en navegador que permite ejecutar consultas SQL, explorar el esquema y administrar la base de datos interactivamente.



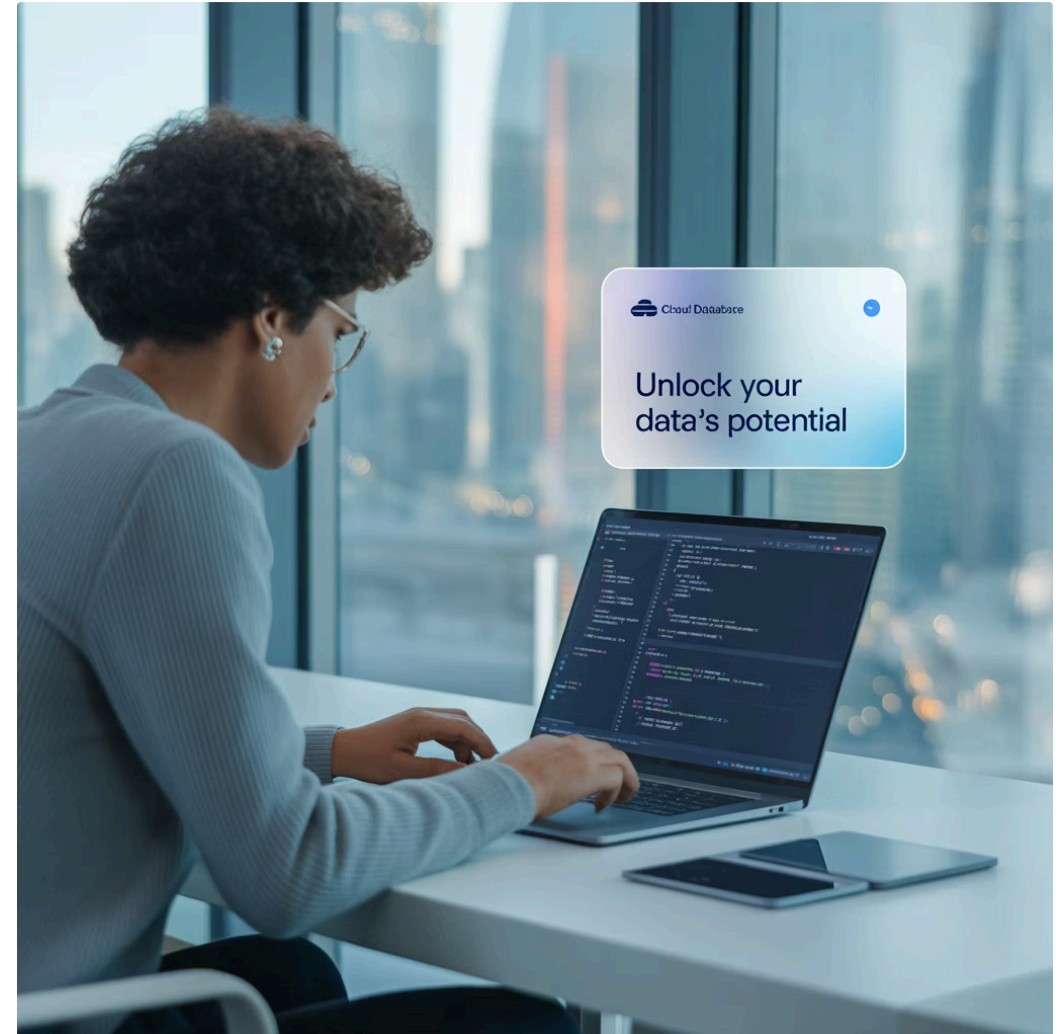
Compatibilidad SQL

Soporta gran parte del estándar SQL-92 y funciones compatibles con otras bases de datos populares (PostgreSQL, MySQL, Oracle).

¿Por qué es Popular H2?

Su ligereza y flexibilidad la hacen perfecta para "levantar" una base de datos rápidamente sin configuraciones complejas, lo cual es invaluable en el ciclo de desarrollo.

- Rápida puesta en marcha
- Mínimo consumo de recursos
- Perfecta para entornos de desarrollo y pruebas
- Fácil integración con aplicaciones Java



Modos de Operación de H2

1

Modo En Memoria

La base de datos reside completamente en la RAM de la JVM. Los datos no persisten una vez que la aplicación se detiene o la conexión se cierra.

Ideal para pruebas unitarias y de integración, donde necesitas una base de datos limpia en cada ejecución.

2

Modo Basado en Archivos

Los datos se almacenan en archivos en el disco duro. Esto permite que la base de datos y sus datos persistan entre reinicios de la aplicación.

Útil para el desarrollo local donde necesitas mantener los datos.

Modos de Operación (continuación)

1

Modo Embebido

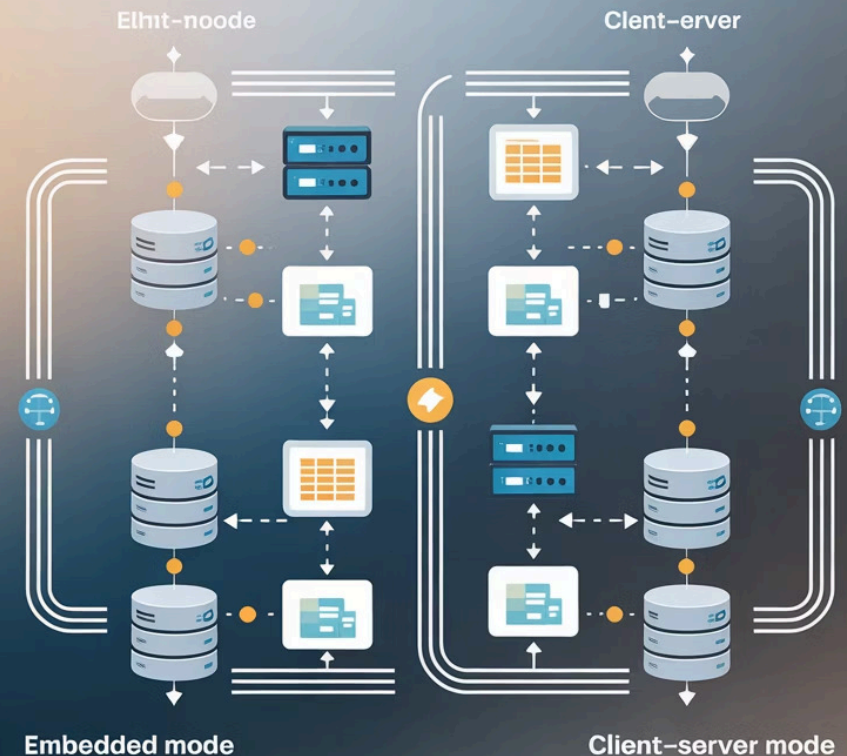
La base de datos se ejecuta dentro de la misma JVM que tu aplicación, sin necesidad de un proceso de servidor separado.

Tu aplicación accede directamente al archivo de la base de datos (o la instancia en memoria).

2

Modo Cliente-Servidor

H2 puede iniciarse como un servidor independiente, permitiendo que múltiples aplicaciones o usuarios se conecten a ella a través de una red usando el protocolo TCP.



Cómo Descargar H2 sin Maven ni Gradle

Visita la Página de Descargas Oficial

Abre tu navegador web y ve a:

<http://www.h2database.com/html/download.html>

Identifica la Versión Estable

Busca la sección "Stable Versions" o "Latest Stable Release".

Descarga el Paquete Binario (ZIP)

Haz clic en el enlace para descargar el archivo ZIP. Por ejemplo, verás algo como h2-2023-12-17.zip (la fecha puede variar).

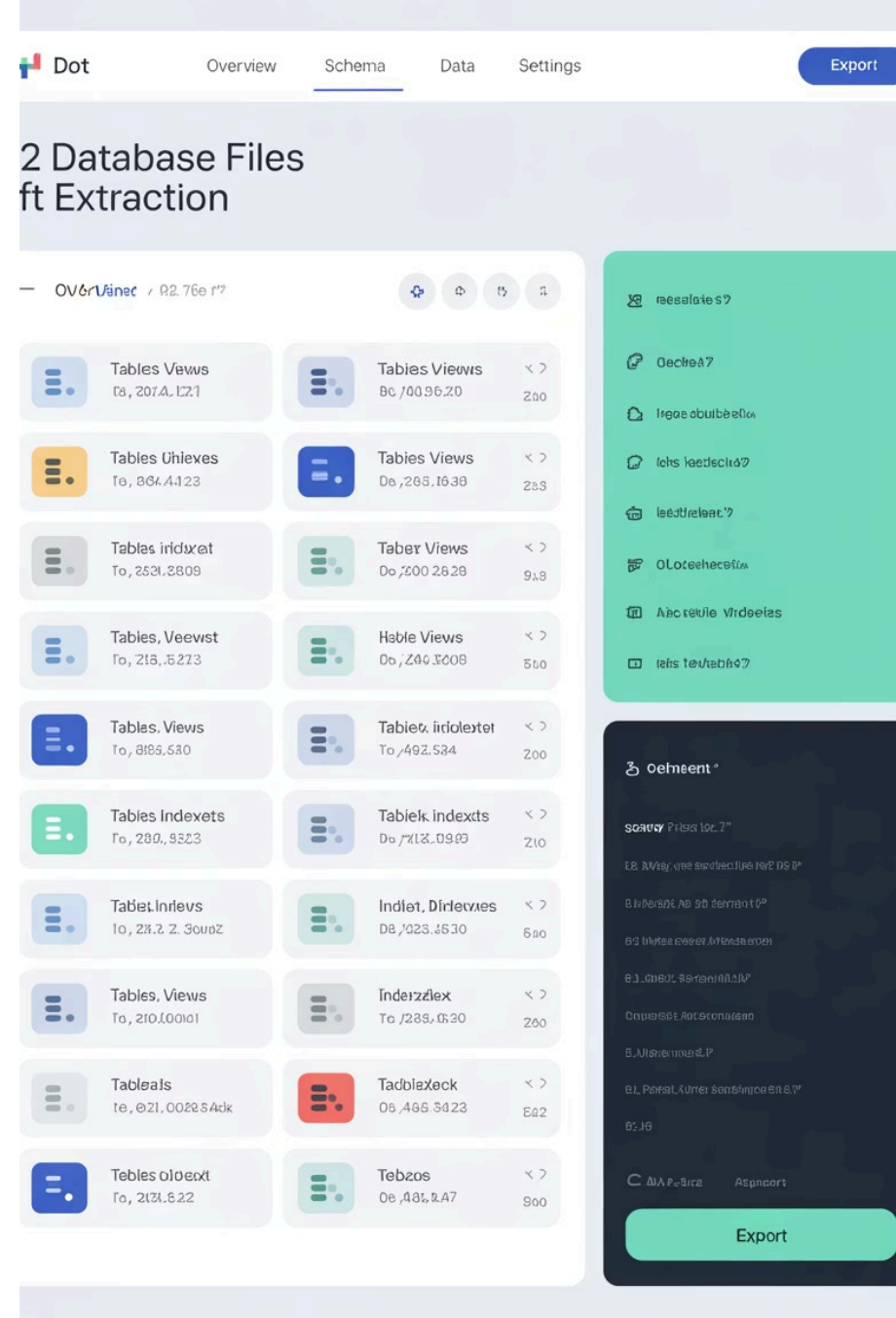
Cómo Descargar H2 (continuación)

Descomprime el Archivo

Una vez descargado, descomprime el archivo ZIP en una ubicación de tu elección en tu sistema de archivos (ej. C:\h2, o /opt/h2 en Linux/macOS).

Localiza el JAR

Dentro de la carpeta descomprimida, encontrarás una estructura similar a h2/bin/. Dentro de bin/, estará el archivo JAR principal de H2 (ej. h2-2.2.224.jar o un nombre similar).





Configuración de H2 con Jakarta Persistence API (JPA)

JPA es una especificación de Java que describe la gestión de datos relacionales en aplicaciones Java. H2 se integra perfectamente con JPA, proporcionando una solución ligera para el desarrollo.

Configurar el Archivo persistence.xml

Este archivo es el corazón de la configuración de JPA. Debe estar ubicado en la carpeta META-INF dentro de tu src/main/resources (o donde tu IDE lo espere).

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
    version="3.0">

    <persistence-unit name="miUnidadPersistenciaH2" transaction-type="RESOURCE_LOCAL">
        <class>com.ejemplo.dominio.Usuario</class>
        <class>com.ejemplo.dominio.Producto</class>
        <properties>
            <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
            <property name="jakarta.persistence.jdbc.url" value="jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1"/>
            <property name="jakarta.persistence.jdbc.user" value="sa"/>
            <property name="jakarta.persistence.jdbc.password" value=""/>

            <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
            <property name="jakarta.persistence.schema-generation.database.action" value="drop-and-create"/>
            <property name="hibernate.show_sql" value="true"/>    <property name="hibernate.format_sql" value="true"/>    </properties>
        </persistence-unit>
    </persistence>

Ejemplo de persistence.xml para H2 (basada en archivos):
XML
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
    version="3.0">

    <persistence-unit name="miUnidadPersistenciaH2File" transaction-type="RESOURCE_LOCAL">
        <class>com.ejemplo.dominio.Usuario</class>
        <class>com.ejemplo.dominio.Producto</class>

        <properties>
            <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
            <property name="jakarta.persistence.jdbc.url" value="jdbc:h2:file:./data/mydb;DB_CLOSE_ON_EXIT=FALSE;AUTO_SERVER=TRUE"/>
            <property name="jakarta.persistence.jdbc.user" value="sa"/>
            <property name="jakarta.persistence.jdbc.password" value=""/>

            <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
            <property name="jakarta.persistence.schema-generation.database.action" value="update"/>
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

Ejemplo de persistence.xml para H2 (en memoria)

```
▼<persistence xmlns="https://jakarta.ee/xml/ns/persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd" version="3.0">
  ▼<persistence-unit name="miUnidadPersistenciaH2" transaction-type="RESOURCE_LOCAL">
    <class>com.ejemplo.dominio.Usuario</class>
    <class>com.ejemplo.dominio.Producto</class>
    ▼<properties>
      <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
      <property name="jakarta.persistence.jdbc.url" value="jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1"/>
      <property name="jakarta.persistence.jdbc.user" value="sa"/>
      <property name="jakarta.persistence.jdbc.password" value=""/>
      <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
      <property name="jakarta.persistence.schema-generation.database.action" value="drop-and-create"/>
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

❗ En este modo, la base de datos existe solo en memoria y se elimina cuando la aplicación se cierra. Ideal para pruebas.

Ejemplo de persistence.xml para H2 (basada en archivos)

```
▼<persistence xmlns="https://jakarta.ee/xml/ns/persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd" version="3.0">
  ▼<persistence-unit name="miUnidadPersistenciaH2File" transaction-type="RESOURCE_LOCAL">
    <class>com.ejemplo.dominio.Usuario</class>
    <class>com.ejemplo.dominio.Producto</class>
    ▼<properties>
      <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
      <property name="jakarta.persistence.jdbc.url" value="jdbc:h2:file:./data/mydb;DB_CLOSE_ON_EXIT=FALSE;AUTO_SERVER=TRUE"/>
      <property name="jakarta.persistence.jdbc.user" value="sa"/>
      <property name="jakarta.persistence.jdbc.password" value=""/>
      <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
      <property name="jakarta.persistence.schema-generation.database.action" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

 En este modo, los datos persisten en archivos en el disco, permitiendo que la información se mantenga entre reinicios de la aplicación.

Código Java para usar JPA con H2

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class JpaH2ConfigExample {
    public static void main(String[] args) {
        // El nombre debe coincidir con el del persistence.xml
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("miUnidadPersistenciaH2");
        EntityManager em = emf.createEntityManager();

        try {
            em.getTransaction().begin();
            // Aquí realizas tus operaciones JPA (persist, find, merge, remove)
            em.getTransaction().commit();
        } catch (Exception e) {
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
            }
            e.printStackTrace();
        } finally {
            em.close();
            emf.close();
        }
    }
}
```

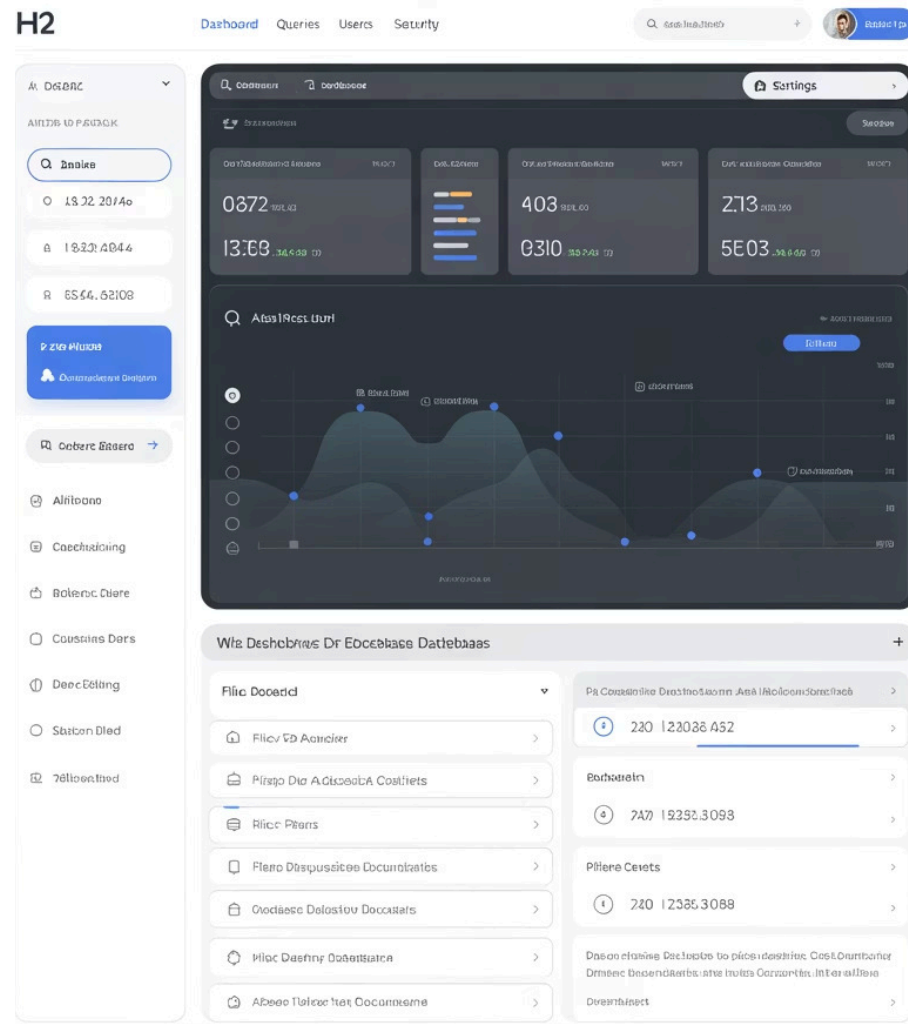
Ejemplo de Operaciones JPA con H2

```
// Crear y persistir un nuevo usuario
Usuario nuevoUsuario = new Usuario("Juan Perez",
"juan.perez@example.com");
em.persist(nuevoUsuario);
System.out.println("Usuario persistido: " + nuevoUsuario);

// Buscar un usuario por ID
Usuario usuarioEncontrado = em.find(Usuario.class, 1L);
if (usuarioEncontrado != null) {
    System.out.println("Usuario encontrado: " +
usuarioEncontrado.getNombre());
}
```



Las operaciones básicas de JPA (crear, leer, actualizar, eliminar) funcionan perfectamente con H2, proporcionando una experiencia de desarrollo fluida.



Configuración de Conexión en la Consola Web

Para Modo en Memoria

Si tu JPA está usando `jdbc:h2:mem:testdb` y tu aplicación está corriendo, en la consola H2, la URL JDBC será:

`jdbc:h2:mem:testdb`

Para Modo Basado en Archivos

Si tu JPA está usando `jdbc:h2:file:./data/mydb` y tiene `AUTO_SERVER=TRUE`, la URL JDBC en la consola H2 será:

`jdbc:h2:tcp://localhost/~/data/mydb`

El usuario por defecto es **sa** y la contraseña está vacía.

Puntos Clave de la Configuración con JPA

persistence.xml

Es el archivo central. Define las unidades de persistencia, las propiedades de conexión a la base de datos y las propiedades del proveedor JPA.

URL JDBC

Define el tipo de base de datos H2 (en memoria o archivo) y su ubicación/nombre.

Driver JDBC

Especifica el driver JDBC a usar (org.h2.Driver).

Generación de Esquema

Importante para que JPA/Hibernate creen o actualicen el esquema de la base de datos automáticamente.

Ventajas de Usar H2 en Desarrollo

- Rápida configuración y puesta en marcha
- No requiere instalación de servidor de base de datos
- Perfecta para entornos de desarrollo y pruebas
- Consola web integrada para administración
- Compatibilidad con múltiples modos de operación
- Excelente integración con JPA y frameworks Java



Resumen: H2 Database con JPA



H2 Database ofrece una solución ligera, rápida y versátil para el desarrollo de aplicaciones Java con JPA, facilitando el ciclo de desarrollo y pruebas.